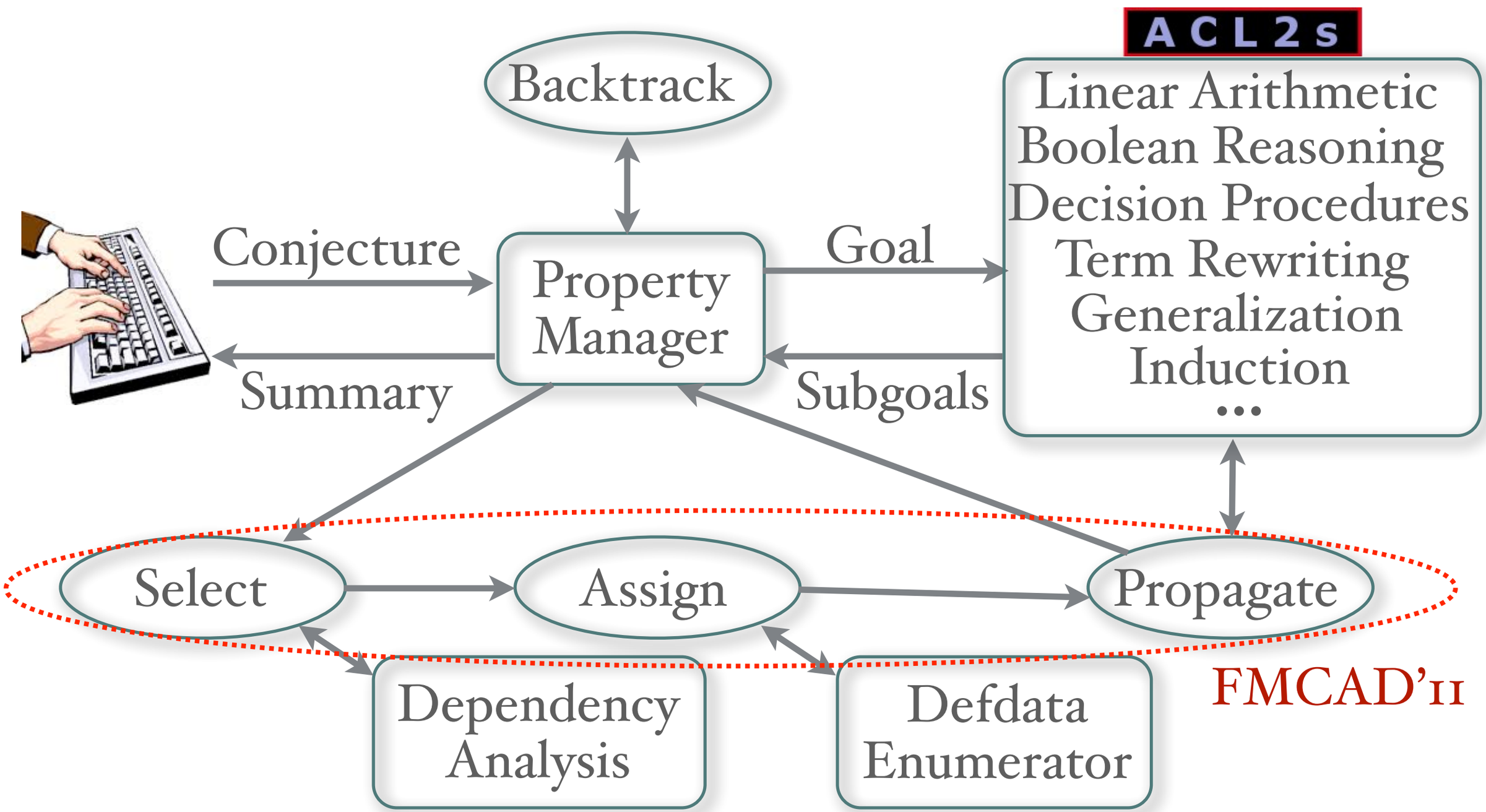
Lecture 18

Pete Manolios
Northeastern

Bird's Eye View of Algorithm

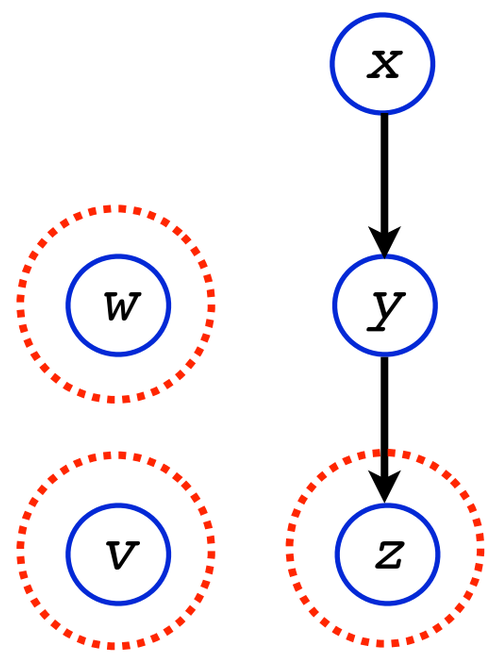


Example: Select

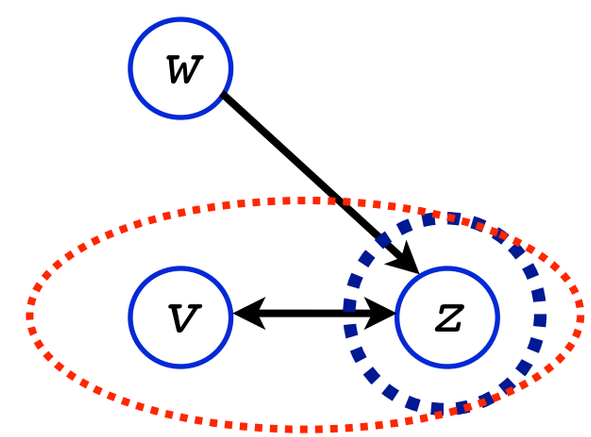
Select

Property P

$x = hash(y)$
$y = hash(z)$
$x+y \neq v^2$
$z > 10$
$w < min(x,y)$
$\Rightarrow$ $w < z$



Equality Graph



Dependency Graph

Example: Assign

Property P

$$
\begin{array}{ll}
 & x = hash(y) \\
 & y = hash(z) \\
 & x+y \neq v^2 \\
 & z > 10 \\
 & w < min(x,y) \\
\Rightarrow & w < z
\end{array}
$$

Assign $\quad z = 34$

Example: Propagate

Property P

```
    x = hash(y)
    y = hash(34)
    x+y ≠ v²
    34 > 10
    w < min(x,y)
⇒   w < 34
```

```
    x = hash(y)
    y = 2689...
    x+2689... ≠ v²
    true
    w < min(x,2689...)
⇒   w < 34
```

```
    x = 3623...
    y = 2689...
    3623...+2689... ≠ v²
    w < min(3623...,2689...)
⇒   w < 34
```

```
    x = 3623...
    y = 2689...
    6313... ≠ v²
    w < 2689...
⇒   w < 34
```

Example: Counterexample

Property P

$x = hash(y)$
$y = hash(z)$
$x+y \neq v^2$
$z > 10$
$w < min(x,y)$
$\Rightarrow w < z$

$z = 34$

Assign & Propagate

$x = 3623...$
$y = 2689...$
$6313... \neq v^2$
$w < 2689...$
$\Rightarrow w < 34$

$x = 3623...$

true
$y = 2689...$
$6313... \neq v^2$
$w < 2689...$
$\Rightarrow w < 34$

$y = 2689...$

true
$6313... \neq v^2$
$w < 2689...$
$\Rightarrow w < 34$

$v = 0$

true
$w < 2689...$
$\Rightarrow w < 34$

$w = 40$

true
$\Rightarrow$ false

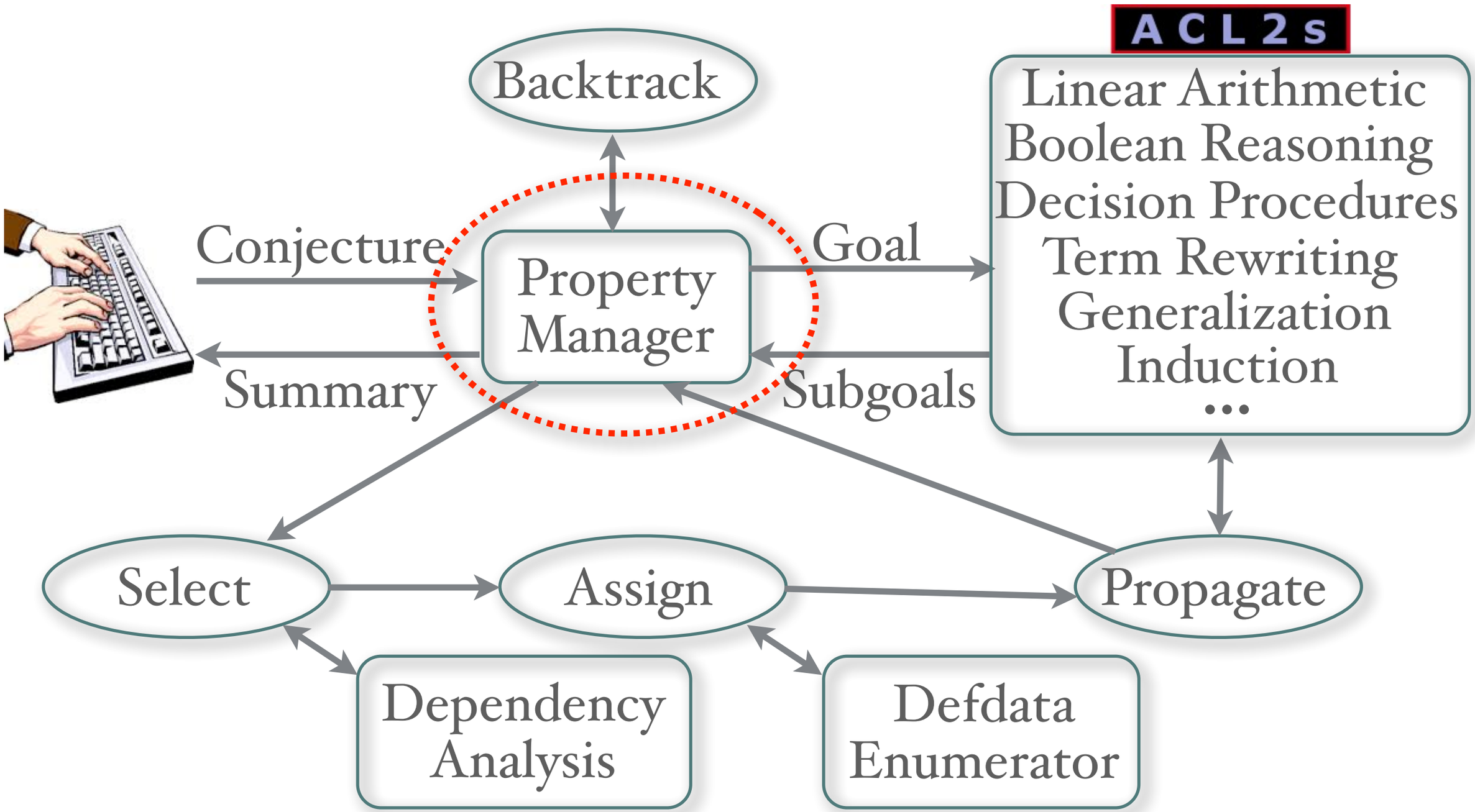
[z = 34
x = 3623...
y = 2689...
v = 0
w = 40]

Counterexample!

Assumptions

- Specification language L
  - Multi-sorted first-order logic
  - Extensible: can define new functions, predicates
  - Executable
- Properties of the form $hyp_1 \wedge \cdots \wedge hyp_n \Rightarrow concl$
  - Universal closure
  - No nested quantifiers
- An Interactive Theorem Prover (ITP) for L
  - `Smash`: takes a goal, returns equi-valid subgoals
  - `Simplify`: takes H (hyps), c returns $c'=c$ under H

Bird's Eye View of Algorithm



Analyze

```
Input:  Property P, Summary summary
Output: Status, Summary of the analysis of P
if P is closed then return AnalyzeConst(P)
n,status := 0,not-done
while ¬StopCond(summary) ∧ n ≤ SLimit do
   A, n := Search(P), n+1
   summary := UpdateA(summary, P, A)
if StopCond(summary) then return (done, summary)
S := SMASH(P)
summary := UpdateS(summary,P,S)
if S ≠ {P} ∧ S ≠ {} then
   for all p ∈ S do
      status, summary := Analyze(p, summary)
      if status = done then return (done, summary)
return (not-done, summary)
```

Constant expressions are easy

Search for a counterexample

SMASH

Recur if SMASH finds structure

Not done

Search

```
Input:  Property P, with at least one free variable
Output: A counterexample (assignment) or fail
local Stack A of (var, val, #decisions, type, prop)
A,d,x := [],0,Select(P)
while true do
    v,t := Assign(x,P)
    if t="decision" then d := d+1
    P′ := Propagate(x,v,P)
    if ¬Vacuous(P′) then
        A := push((x,v,d,t,P), A)
        if A is complete then return A
        d,P,x := 0,P′,Select(P′)
    else if A ≠ [] then
        repeat
            (x,_,d,t,P) := head(A)
            A := pop(A)
        until (t="decision" ∧ d≤BLIMIT) ∨ A=[]
    if A=[] ∧ (t="implied" ∨ d>BLIMIT) then return fail
```

Establish invariant

Assign & Propagate

Extend the assignment

Search Failure

Backtrack

Select

```
Input:  Property P, with at least one free variable
Output: A free variable in P
 1: if ∃h∈hyps(P) of the form x=c then return x
 2: G=,P' := buildEqualityGraph(P, vars(P))
 3: S := sink components of SCC(G=)
 4: Sinks := choose x from each s ∈ S
 5: G⋈ := buildDependencyGraph(P', Sinks)
 6: D⋈ := SCC(G⋈)
 7: I := a sink ∈ D⋈ with maximum Imp=, minimum cardinality
 9: return x∈I with maximum Imp= value
```

Constant propagation

Equality graph

Dependency graph

Select variable with most implications

Further Aspects of Algorithm

- ACL2s is untyped: we provide a data definition facility
  - records, union, product, mutually recursive, custom, ... types
  - automatically generate enumerators, used to assign values
- Maintain a global type graph
  - maintains subtype, disjointness information
  - constructed & extended only by proving theorems
- We infer the minimal types of a variable
  - Eg: From ... $x \in \mathbb{Q} \wedge x > 2/7 \wedge x \in \mathbb{Z}$ ..., we infer $x \in \mathbb{N}^+$
  - Requires care because our theorem prover is not monotonic
- Modes: Incremental & simple, different sampling methods, ...
- Efficiency considerations: we generate a program that generates tests and compile it, and we do it per subgoal

Expert Example

- An expert who proves theorems using ACL2 for a living
- Here is a high school algebra problem that I've been staring at for a couple of hours and is making me feel pretty stupid

  Let a, b, and c be positive rationals

  If $a^2 \leq b*(c+1)$ and $b \leq 4*c$

  Then $(a-1)^2 < b*c$

- Gave an informal proof and asked for an ACL2 proof
- **DEMO**

Software: Comparison with Alloy

- Alloy
  - Declarative, high-level modeling language
  - Based on sets, relations, transitive closure
  - Used to analyze high-level models of software
- Automated analysis
  - Given *scope*: bound on # model elements
  - Generates Boolean formula
  - Shipped to SAT solver

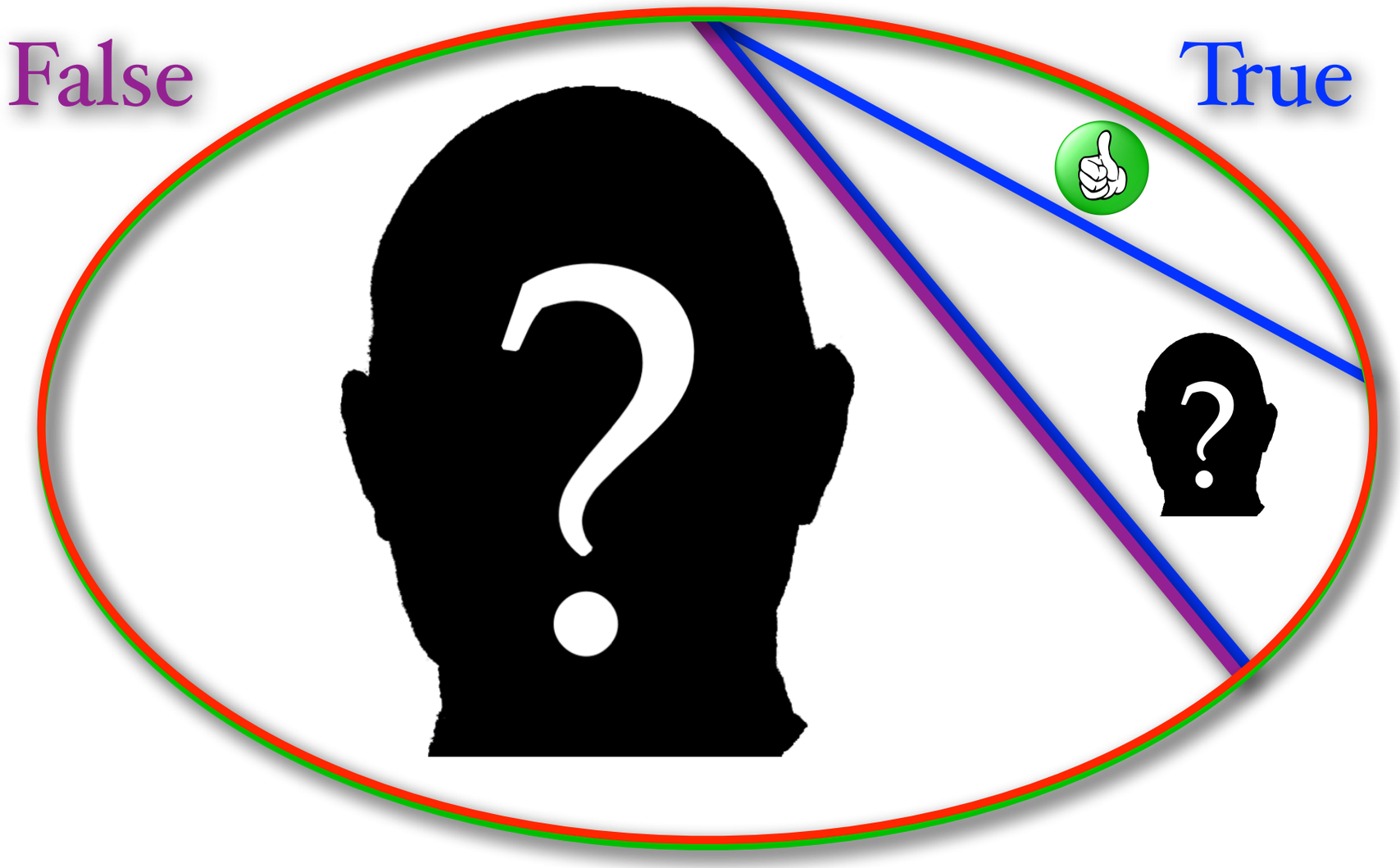
Software: Comparison with Alloy

Property	Alloy Analyzer			Our method	
	Scope	Time	Result	Time	Result
delUndoesAdd	25	26.41	–	0.07	QED
addIdempotent	25	37.76	–	0.19	QED
addLocal	3	0.08	CE	1.35	CE
lookupYields	3	0.05	CE	0.83	CE
writeRead	34	99.69	–	0.02	QED
writeIdempotent	33	44.13	–	0.01	QED
hidePreservesInv	61	24.91	–	0.26	QED
cutPaste	3	0.20	CE	0.49	CE
pasteCut	3	0.20	CE	1.38	CE
pasteAffectsHidden	27	117.63	–	0.42	QED
markSweepSound	8	47.34	–	0.28	QED
markSweepComplete	7	58.12	–	0.34	QED

Software: Comparison with Alloy

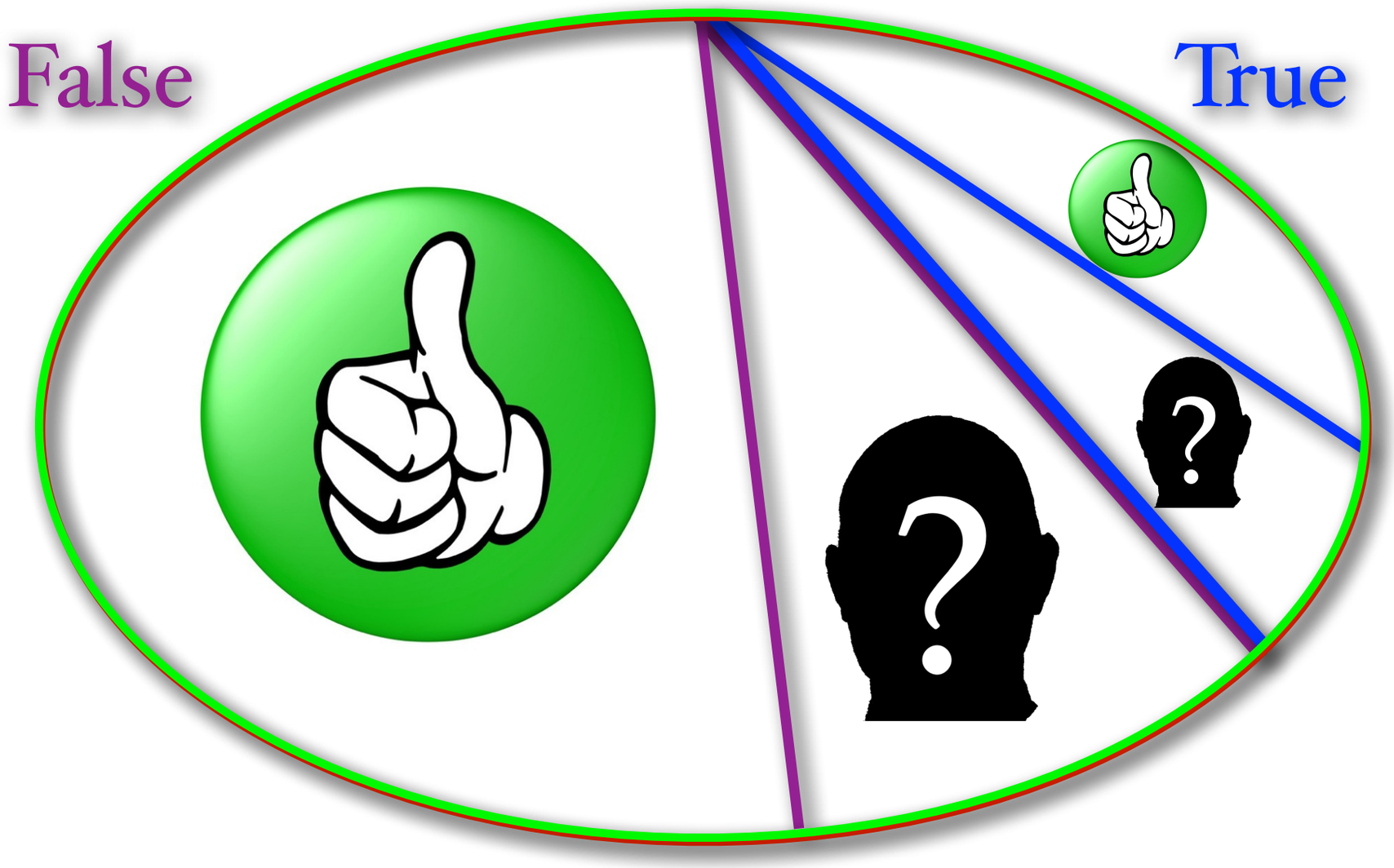
- Models in ACL2 mimicked Alloy formalization
- Used *set* and *map* types
- The libraries in ACL2 *proved* all properties Alloy posits
- No lemmas, hints, user guidance was needed
- Highlights effectiveness of domain-specific libraries

ITP Experience



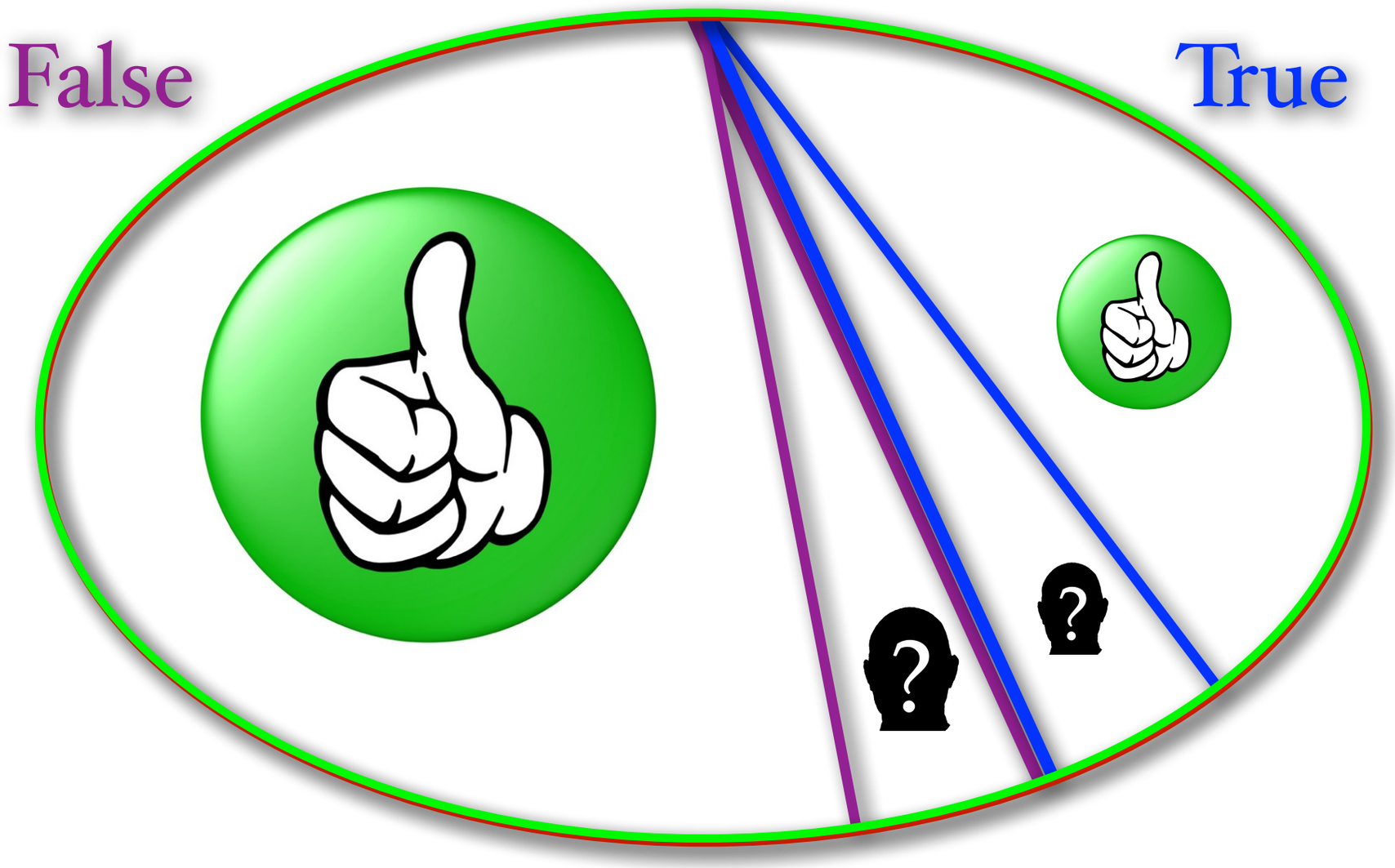
Conjectures

ACL2s Experience



Conjectures

LLM Experience?



Conjectures

Future Work

- Counterexample synthesis: efficient, parameterized functions to quickly generate potential counterexamples after theorem proving decomposition
- Defdata types: expressiveness, algebras, decidability
- Interactive counterexample generation: allow regular engineers to guide counterexample generation
  - Find, report bottlenecks (e.g., hyps not satisfied)
  - Fixers (e.g., remove-dups is a fixer for no-dups)
- *Evaluate model-coverage wrt a set of properties, implementation*
- Integrate decision procedures: automate handling of subgoals in decidable fragments, dispatch unsatisfiable subgoals quickly using sound abstractions, extend with algorithm for better SAT capability
- Support for constrained functions (via encapsulate, quantifiers)
- LLM integration